

Parallel Morphological Opening

Performance Analysis of Image Processing with OpenMP in C++

Pere Llauradó Adeva | Mid Term Project Presentation

Problem Introduction

Context & Motivation

I solved the **Opening** operation (Erosion + Dilation) used for:

- **Medical Imaging:** Detecting diabetic retinopathy by cleaning noisy fundus images.
- **Earth Observation:** Smoothing contours of water bodies in satellite imagery.
- **The Goal:** Dramatically reduce high execution times for massive datasets.



Computational Challenge



Complexity

9x9 structural element requires visiting 81 neighbors for **every single pixel**.



Memory Bound

Billions of memory reads. Data volume often exceeds the high-speed L3 cache capacity.



Parallelization costs

For small datasets, thread preparation time represents a larger proportion of the total execution time.

The Approach

Opening: A Two-Phase Algorithm

1. Erosion Phase

Scans the image and replaces each pixel with the **minimum** intensity in the neighborhood.

Effect: Shrinks objects, removes bright noise.

2. Dilation Phase

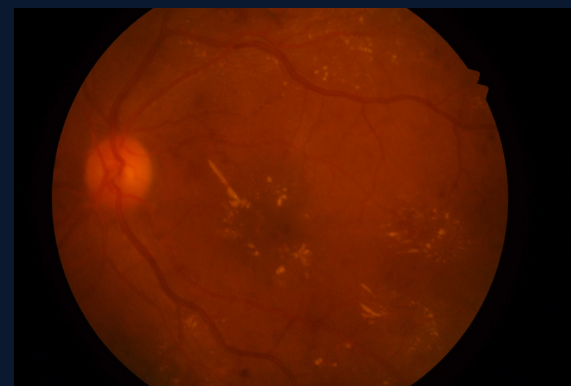
Reverses the process using the eroded image. Replaces pixels with the **maximum** value.

Effect: Restores object size, keeps it clean.

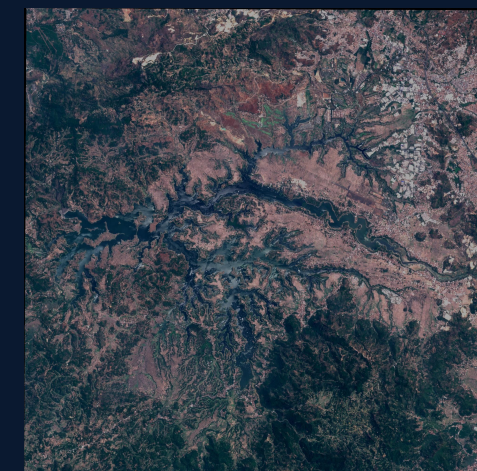
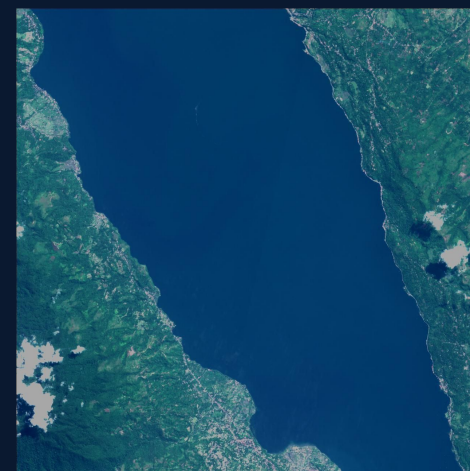
Constraint: A synchronization barrier is required between phases.

Datasets used

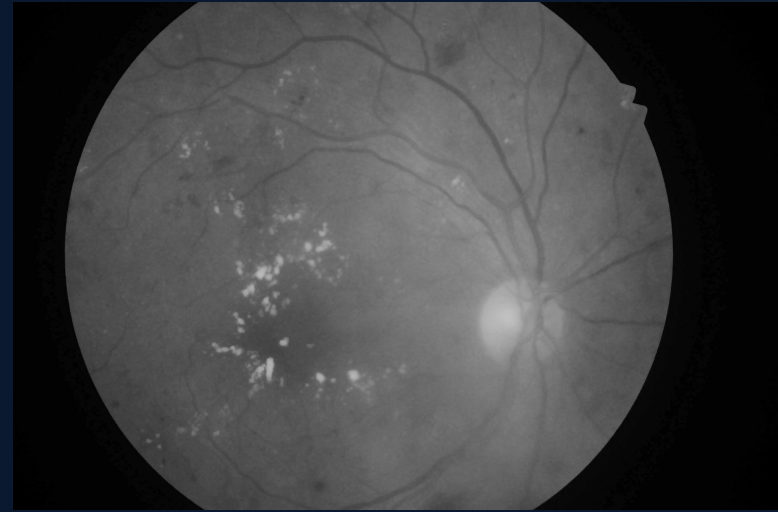
Indian Diabetic Retinopathy Image Dataset: RGB images with 4288×2848 pixels resolution



Satellite Images of Water Bodies: RGB images with an image resolution between 256×256 to 1024×1024 px.



Opening: Visual results



Parallelization Strategy

Strategy

Data Parallelism: Pixels are independent within each phase.

Collapse Loops: I flattened the two outer loops (representing the height and width of the image) into a single iteration space.

Scheduling strategies: trying dynamic vs static openMp strategies to compare the performance

Race conditions: avoid race conditions to avoid more complex operations

Key optimizations

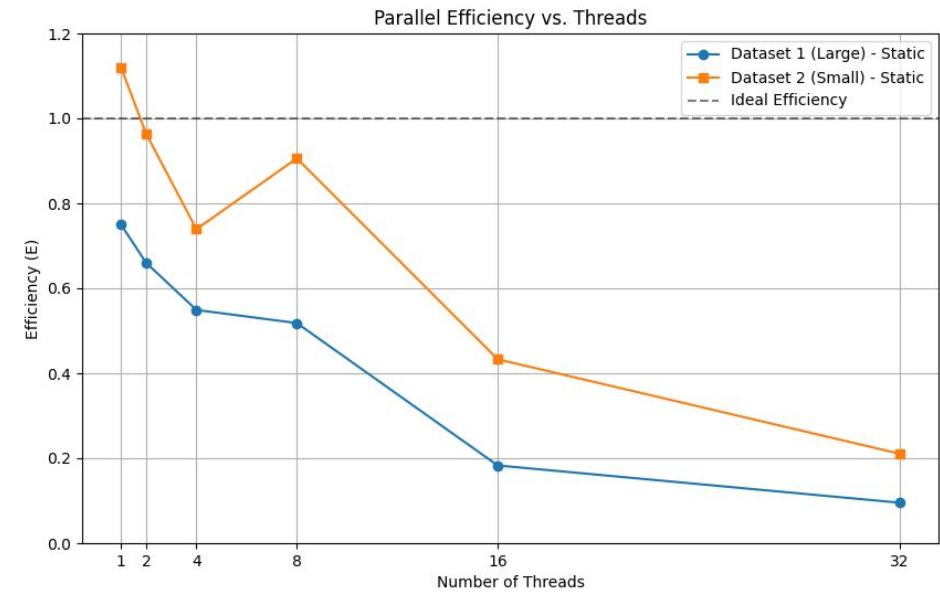
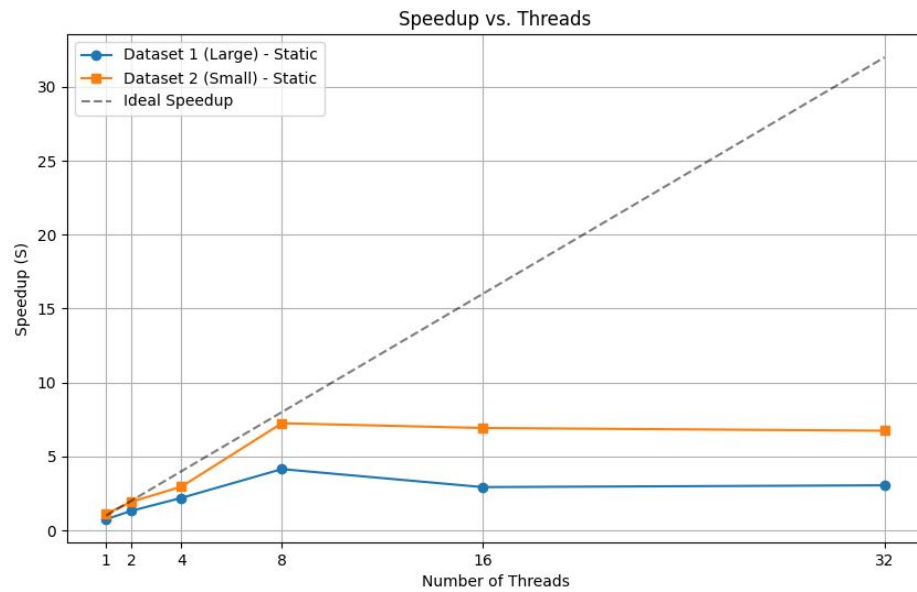
Key Optimizations:

- **Data Parallelism:** Pixels are independent within each phase.
- **Collapse(2):** Merges Height and Width loops to expand the iteration space.
- **Buffer Separation:** Uses separate I/O arrays to eliminate Race Conditions.
- **Memory Warm-up:** Pre-loading data into cache before benchmarking.
- **1D Vectors:** width * height size vectors → 1D vector instead of 2D.

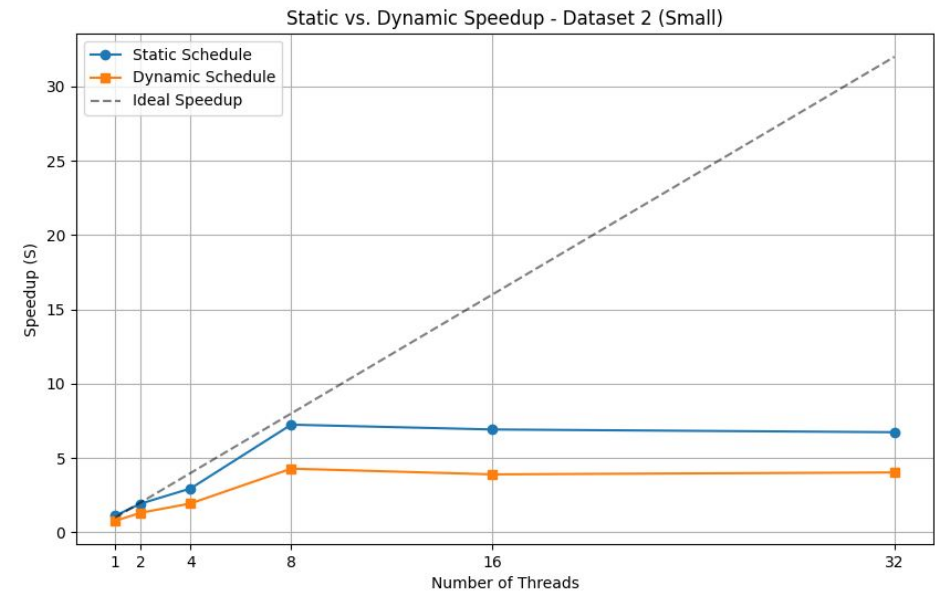
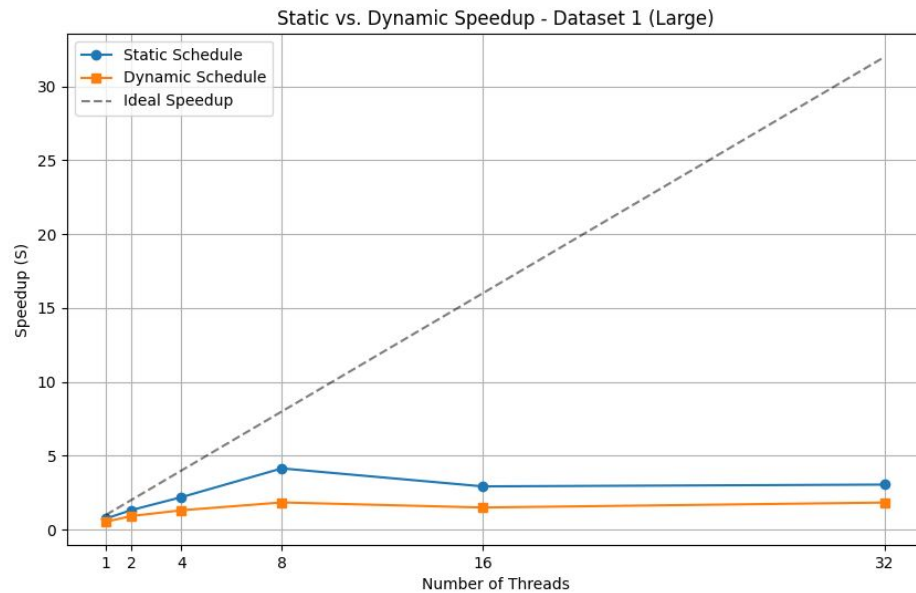
```
#pragma omp parallel for collapse(2)
```

Performance Results

Plots



Plots: static vs dynamic speedup



Best configuration

Hw settings: i7-8550U with 4 physical cores and 8 logical processors utilizing Hyper-Threading

Threads: 8 is the best number of threads for both

Scheduling: static is the best scheduling method

Dataset 1 (Medical)

Large Retina images. High RAM bandwidth usage.

4.2x Speedup

Limited by the **Memory Wall**.

Dataset 2 (Satellite)

Smaller Water Body images. Fits in L3 Cache.

7.25x Speedup

Achieved near-ideal scaling!

Interesting findings or surprises

The memory wall

Expected → **Gustafson's Law**: larger workloads should scale better as parallel work dominates overhead.

Memory Bound → The 9x9 kernel 81 memory reads for each pixel

Cache misses: large dataset exceeds cache capacity → Cache misses

Conclusion: the CPU cores spend more time waiting for data to arrive than actually processing pixels.

Conclusions & Lessons

Summary

Performance gains

- **Speedup:** peak speed up of 7.25x and 4.14x
- **Perfect results:** 100% pixel-perfect validation across all parallel configurations compared to the sequential baseline.

Memory wall

- **Memory-Bound:** Performance is limited by memory bandwidth rather than raw CPU power. once data exceeds cache capacity, scaling is capped by RAM speed.

Optimal Config

- **8 Threads:** Matches logical cores.
- **Scheduling:** Static gives the best results
- **Cache Awareness:** Dataset size determines scaling behavior.

Hardware Architecture Awareness:

- **Cores:** 4 physical cores capacity. 8 logical cores with hyperthreading

Lessons learned

Hardware awareness: logical cores are more efficient than Hyper-threading 8 logical cores

Dataset size matters: if data is too big that doesn't fit in the cache the performance will fall and our results won't follow Gustafson's law

Simple is often better: for simple tasks static is better than dynamic

Thank You

Any Questions or Discussion
points?

✉ pere.laurado@unifi.it

🐙 github.com/PereLlauradoAdeva

References

- [1] AaryaPatel98. Indian Diabetic Retinopathy Image Dataset. Kaggle Dataset Repository URL.
<https://www.kaggle.com/datasets/aaryapatel98/indian-diabetic-retinopathy-image-dataset/code/discussion>
- [2] Francisco Escobar. Satellite Images of Water Bodies. Kaggle Dataset Repository URL.
<https://www.kaggle.com/datasets/franciscoescobar/satellite-images-of-water-bodies/discussion?sort=hotness>
- [3] Parallel Programming Course Lecture Notes and Slides. Course material from the Parallel Programming course.